

A Few Examples of Dax Formulations I Find Interesting/Useful

Since Power BI is much more than a click-and-drag program....but also a mini-database with two coding languages. The purpose of this document is to give you a glimpse into what is happening behind the bar and line charts.

In no particular order...

Example 1:

This makes a dynamic date range table based on words, such as “last few months,” and then has a slicer based on it. The last two rows in particular are very unintuitive IMO, since you can’t just use time intelligence functions like ENDOFQUARTER when making a table. Of course, one could make a table that is simply the word “last quarter” and then include the ENDOFQUARTER in the CALCULATE statement. I found this a bit cleaner though, since you can look at the dates in the table to verify they are correct, separate from other DAX.

```
1 margin time period ref table =
2 --TRYING TO DO THIS THE TRADITIONAL WAY WITH DATATABLE DOESN'T WORK BECAUSE time functions
3 --don't work in DATATABLE('...
4 ---decided to make it month ends and beginnings so I can make formulas < or >. Cleaner IMO than >=
5 UNION
6     ROW ( "Date Range", "Last three months", "Period Start", EOMONTH ( TODAY (), -4 ), "Period End", TODAY() ),
7     ROW ( "Date Range", "YTD", "Period Start", DATE ( YEAR ( TODAY () ) -1, 12, 31 ), "Period End", TODAY() ),
8     ROW ( "Date Range", "Last calendar year", "Period Start", DATE ( YEAR ( TODAY () ) -2, 12, 31 ), "Period End", DATE ( YEAR ( TODAY
9     ) ), 1, 1 ) ),
10    ---woah, would love it if we could just use startofquarter/endofquarter with today()
11    ROW ( "Date Range", "Last Quarter", "Period Start",DATE( YEAR( TODAY()), ( ROUNDDOWN( MONTH( TODAY() ) / 3, 0 ) * 3 ) - 2, 1)-1,
12    "Period End", DATE( YEAR( TODAY()), ( ROUNDUP( MONTH( TODAY() ) / 3, 0 ) * 3 ) - 2, 1) )
13
```

You can then just reference these dates in your greater than/less than statements when doing math:

```
1 Dynamic_time_range_Margin =
2 --the below looks like it has too many extra steps, but for whatever reason, DAX
3 ---doesn't recognize VAR thing = column without it being wrapped in math functions sometimes
4 Var Start_Period = SELECTEDVALUE('margin time period ref table'[Period Start])
5 Var End_Period = SELECTEDVALUE('margin time period ref table'[Period End])
6 VAR netprofit = CALCULATE(SUM('Profit_Tax Calculation'[Net_Profit]),'Profit_Tax Calculation'[Start_of_month]>Start_Period,
7 'Profit_Tax Calculation'[Start_of_month]<End_Period)
8 VAR totalrev = CALCULATE(SUM('Profit_Tax Calculation'[All_Revenue]),'Profit_Tax Calculation'[Start_of_month]>Start_Period,
9 'Profit_Tax Calculation'[Start_of_month]<End_Period)
10 RETURN DIVIDE(netprofit,totalrev,0)
```

Example 2:

I wanted a table made up of many other tables, without referencing/merging tables since it also included math (summing by year) and renaming column headers.

```
1 Income_Statement =
2 UNION
3 SUMMARIZE('Expenses (2) for profit and tax table relationship',
4 'Expenses (2) for profit and tax table relationship'[Expense Year],
5 'Expenses (2) for profit and tax table relationship'[Expense Type],
6 "Amount",-SUM('Expenses (2) for profit and tax table relationship'[Amount]))
7
```

```

SUMMARIZE(
    'PAYROLL_TABLE',
    'PAYROLL_TABLE'[Payroll_year],
    "Payroll Type",      -- This defines the text string column name
    "Payroll",           -- This is the static text value assigned to that column
    "Amount",            -- This defines the amount column name
    -SUM('PAYROLL_TABLE'[Gross_Payroll_Amount]) -- Summarize requires an aggregation function for new columns
),

SUMMARIZE('Monthly Subs & churn Data, Recurring Rev Pulled in',
'Monthly Subs & churn Data, Recurring Rev Pulled in'[Year],
"Revenue Type",
"Revenue",
"Income",
sum('Monthly Subs & churn Data, Recurring Rev Pulled in'[Total_Revenue_ALL_Sources])),

SUMMARIZE('DIVIDEND_Table',
DIVIDEND_Table[Year],
"Expense Type",
"Dividends",
"Amount",
-sum('DIVIDEND_Table'[Dividend_Payout])),

```

I think this formulation is also easier to read than a bunch of reference tables. You can see where any error is coming from.

Example 3:

The Dax can be hard to grasp but the situation is clearer: I wanted one visual to filter based on another visual, but there was more than one condition (same stocks, after a date, and before a date), so I couldn't make the connection with one filter on the visual or a relationship in the semantic layer. In this case, I made the criteria variables, then did a return statement to do math based on said criteria, then didn't even use the math, I just added it as a filter on the new second chart.

```

1 filter_measure_for_PG_Page =
2 VAR Opendate_inquestion = MIN('Aggregated, actual trades held > 2 days, PG ONLY'[Avg_Open_Date])
3 VAR End_date_in_question = MAX('Aggregated, actual trades held > 2 days, PG ONLY'[Closed Date])
4 VAR Ticker_in_question = SELECTEDVALUE('Aggregated, actual trades held > 2 days, PG ONLY'[Symbol])
5
6 RETURN
7 SUMX(
8 IF('combines RSI and TOD Strategy'[Ticker] = Ticker_in_question &&
9     'combines RSI and TOD Strategy'[Trading_date] >= Opendate_inquestion &&
10     'combines RSI and TOD Strategy'[Trading_date] <= End_date_in_question,1,BLANK()))
11

```

Example 4:

I needed to “v-lookup” between tables but didn’t know if I was looking it up based on today, tomorrow, the following Monday, the following Tuesday if it was a three-day weekend. So we needed to add conditions to the if statement:

```
1 Open_Next_Trading_Day =
2 VAR Start_of_real_trade = 'Aggregated, actual trades held > 2 days, PG ONLY'[Avg_Open_Date]
3 VAR next_day_after_real_trade = Start_of_real_trade + 1
4 VAR two_days_later = Start_of_real_trade + 2
5 VAR three_days_later = Start_of_real_trade + 3
6 VAR pulling_date = LOOKUPVALUE('PG STOCK Extra stats for example'[Date], 'PG STOCK Extra stats for example'[Date], Start_of_real_trade)
7
8 --lets look up price for each date
9
10 VAR price_day_after = LOOKUPVALUE('PG STOCK Extra stats for example'[Open], 'PG STOCK Extra stats for example'[Date],
    next_day_after_real_trade)
11 VAR price_2days_after = LOOKUPVALUE('PG STOCK Extra stats for example'[Open], 'PG STOCK Extra stats for example'[Date], two_days_later)
12 VAR price_3day_after = LOOKUPVALUE('PG STOCK Extra stats for example'[Open], 'PG STOCK Extra stats for example'[Date], three_days_later)
13
14 --now we pick first non-blank
15
16 RETURN SWITCH(TRUE(), NOT(ISBLANK(price_day_after)), price_day_after, NOT(ISBLANK(price_2days_after)), price_2days_after, price_3day_after)
17
18 |
```

Example 5:

A plain old running total. This formulation still fascinates me in a way because it does just look like it should make a simple sum. But it does indeed do a running sum

```
1 Amount running total =
2 VAR CurrentDate = MAX('Combined one-time and recurring revenue'[Invoice Date])
3 VAR CurrentCustomer = MAX('Combined one-time and recurring revenue'[Customer Code])
4
5 RETURN
6 CALCULATE(
7     SUM('Combined one-time and recurring revenue'[Amount]),
8     FILTER(
9         ALLSELECTED('Combined one-time and recurring revenue'),
10         'Combined one-time and recurring revenue'[Customer Code] = CurrentCustomer &&
11         'Combined one-time and recurring revenue'[Invoice Date] <= CurrentDate
12     )
13 )
```

Example 6:

This one was good for making a payroll calendar. It makes a list of all dates between a date in the past and today and says whether it’s the last day or the month or not. Then the next step is: pull only dates that are the 15th of the month, or the end of the month.

That's your payroll calendar.

```
1 PAYROLL_TABLE =
2 VAR StartDate = DATE(2019, 1, 1)
3 VAR EndDate = TODAY()
4 VAR DateList =
5     ADDCOLUMNS(
6         CALENDAR(StartDate, EndDate),
7         "DayNum", DAY([Date]),
8         "IsEndOfMonth", [Date] = EOMONTH([Date], 0)
9     )
10 RETURN
11     SELECTCOLUMNS(
12         FILTER(
13             DateList,
14             [DayNum] = 15 || [IsEndOfMonth] = TRUE()
15         ),
16         "Date", [Date]
17     )
```